

2. LE TRI-FUSIONa) Principe

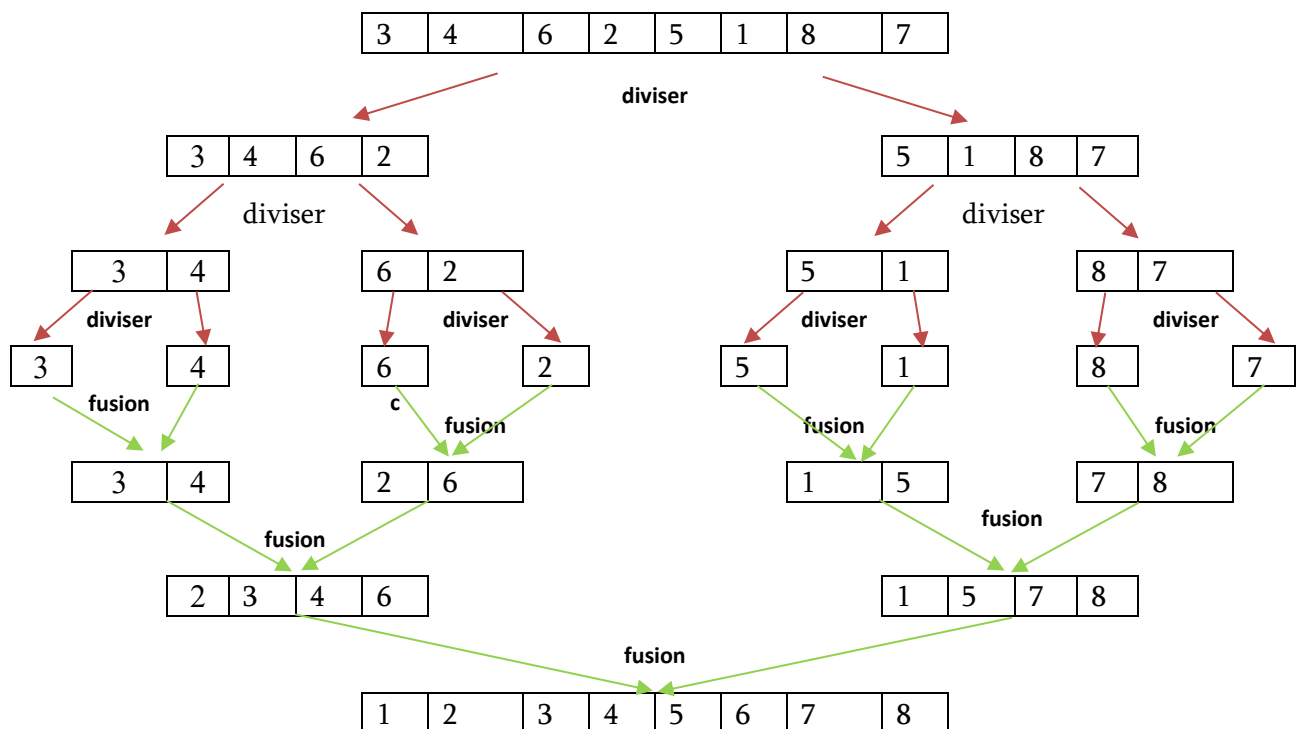
Le tri fusion consiste à trier récursivement les deux moitiés de la liste, puis à fusionner ces deux sous-listes triées en une seule. La condition d'arrêt à la récursivité sera l'obtention d'une liste à un seul élément, car une telle liste est évidemment déjà triée.

Voici donc les trois étapes (diviser, régner et combiner) de cet algorithme :

- ✚ Diviser la liste en deux sous-listes de même taille (à un élément près) en la "coupant" par la moitié.
- ✚ Trier récursivement chacune de ces deux sous-listes. Arrêter la récursion lorsque les listes n'ont plus qu'un seul élément.
- ✚ Fusionner les deux sous-listes triées en une seule.



Ce schéma illustre le processus :



Rq1. On ramène donc le problème d'une liste aux sous-problèmes du tri de deux listes deux fois plus petites, jusqu'à parvenir à des listes d'au plus un élément sur lesquels le tri est bien sûr déjà fait.

b) Implementation

La remarque énoncée ci-dessus permet d'envisager le tri fusion en Python sous la forme suivante (à condition d'écrire la fonction **diviser(lst)**) :

```
def diviser(lst):
    """prend une liste lst et
    renvoie deux listes correspondant à la moitié droite et à la moitié gauche"""
    pass
```

```
def tri_fusion (lst):
    if len(lst)<2 :
        return lst
    left,right=diviser(lst)
    print(left,right)
    return fusion_rec(tri_fusion(left),tri_fusion(right))
```



[tri_fusion.py](#)

Exo 2.

- Implémenter ce tri, tester.
- Que se passe-t-il si l'on cherche à trier une liste de 1000 éléments ?
- D'après les résultats obtenus l'an dernier, on sait que le coût d'un tri par sélection est quadratique (quel que soit le tableau à trier) et le coût d'un tri par insertion est quadratique dans le pire des cas. Quelle est la signification d'une telle affirmation ?
- Si k est le nombre d'étapes de divisions, et n la taille du tableau à trier, compléter l'inégalité :
 $2^{\dots} < \dots \leq 2^{\dots}$
- Compléter en indiquant le coût (nombre de comparaisons nécessaires à la fusion) dans chaque niveau de l'arbre :

			coût majoré par ...
		1 fusion de 2 listes de tailles $n/2$	$2 \times \frac{n}{2} =$

La complexité du tri fusion est en $O(n \log n)$

Rq2. C'est la meilleure complexité pour un tri par comparaisons. Or en l'absence d'informations a priori sur les valeurs à trier, les tris par comparaisons sont les seuls envisageables. L'efficacité d'un tri fusion est donc optimale dans ce cas le plus général.

Rq3. Néanmoins en raison des découpages répétés, l'algorithme nécessite de l'espace mémoire supplémentaire qui peut être un facteur critique lorsque le jeu de données à trier est conséquent.

- Compléter les mesures de performance sur votre PC exécutés par le programme **tri_fusion.py** :

Taille n du tableau	Temps d'exécution tri selection	Temps d'exécution tri fusion
200		
400		
800		
1600		
3200		
6400		
12800		

Exo 3. la taille du tableau est $n = 1000$, on a relevé les temps d'exécution suivants

taille	tri selection	tri fusion
1000	0,06 s	0,01 s

- Le temps d'exécution d'un tri par sélection est proportionnel à
- Le temps d'exécution d'un tri fusion est proportionnel à
- Compléter le tableau suivant :

taille	tri sélection – temps d'exécution approximatif	tri fusion – temps d'exécution majoré par
1000	0,06 s	0,01 s
2000		
4000		
1000000		

II. LA STRATEGIE « DIVISER POUR REGNER »

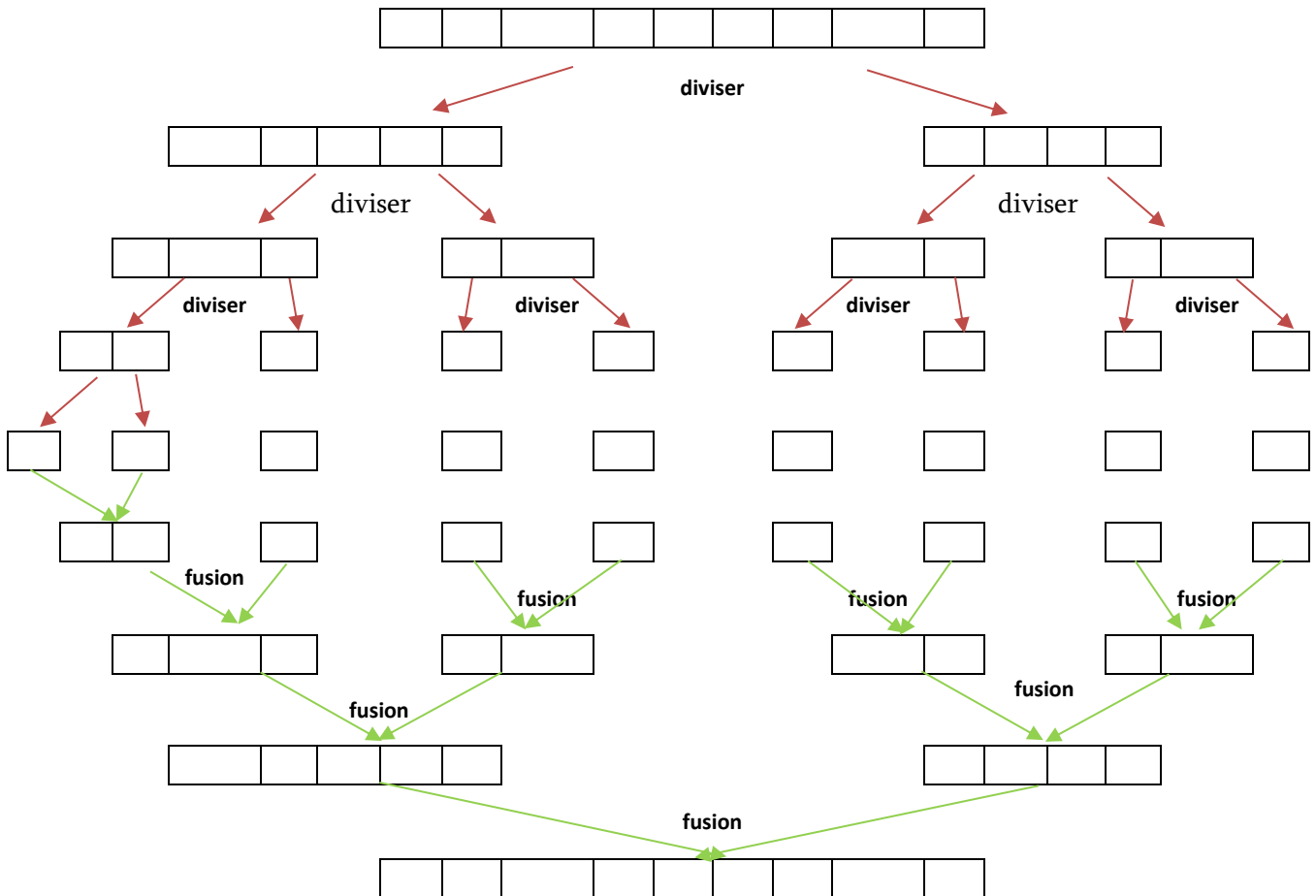
La stratégie « diviser pour régner » (« *divide and conquer* » en anglais) en trois étapes :

- Diviser le problème en sous-problèmes (de préférence **nettement plus petits**)
- Résoudre (généralement **récursivement**) ces sous-problèmes
- Combiner les solutions pour reconstruire la solution du problème original

Rq4. Stratégie principalement utilisée pour obtenir de meilleures complexités que celles données par un algorithme moins évolué.

III. EXOS (DIVISER POUR REGNER)

Exo 4. Compléter le schéma suivant qui décrit le tri-fusion de la liste [23,12,4,56,35,32,42,54,3] de taille 9



Exo 5. Recherche dichotomique.

Le problème est de rechercher la présence d'un élément v dans une liste t préalablement triée.

L'idée pour résoudre cette question est d'utiliser une méthode dichotomique.

La liste étant triée, après comparaison avec l'élément du "milieu" il est en effet facile de voir dans quelle moitié peut éventuellement se trouver l'élément cherché. On aura plus alors qu'à recommencer récursivement la recherche.

Le terme dichotomie provient du grec ancien "*dikhotomia*", signifiant "division en deux parties".

les trois étapes de la résolution de ce problème illustration de la méthode "diviser pour régner" :

1. Diviser la liste t en deux sous-listes en la "coupant" par la moitié.
2. Rechercher récursivement la présence de l'élément recherché v dans la "bonne" des deux sous-listes après l'avoir comparé à l'élément situé au milieu de la liste.
3. Renvoyer la valeur d'un indice où se trouve la valeur v recherchée dans la liste t fournie en paramètre.

Ecrire l'algorithme en pseudo-code et implémenter le programme *Python* correspondant qui réalise une recherche dichotomique dans un tableau trié. (On pourra compléter le fichier **recherche_dichotomique tbc.py**)



[recherche_dichotomique.py](#)

Exo 6. Recherche du max

Le problème est de rechercher le maximum d'une liste de nombres.

En adoptant la stratégie "*diviser pour régner*", l'idée pour résoudre cette question est de rechercher récursivement le maximum de la première moitié de la liste et celui de la seconde, puis de les comparer. Le

plus grand des deux sera le maximum de toute la liste. La condition d'arrêt à la récursivité sera l'obtention d'une liste à un seul élément, son maximum étant bien sûr la valeur de cet élément.

les trois étapes de la résolution de ce problème illustration de la méthode "diviser pour régner" :

- ✚ Diviser la liste en deux sous-listes en la "coupant" par la moitié.
 - ✚ Rechercher récursivement le maximum de chacune de ces sous-listes. Arrêter la récursion lorsque les listes n'ont plus qu'un seul élément.
 - ✚ Retourner le plus grand des deux maximums précédents.
1. Ecrire l'algorithme en pseudo-code et implémenter le programme *Python* correspondant qui réalise cette recherche de maximum.
 2. Déterminer la complexité de cette recherche. Est-elle préférable à une recherche par balayage ?



[recherche_du_max.py](#)

Exo 7. Rotation d'image (quart de tour)

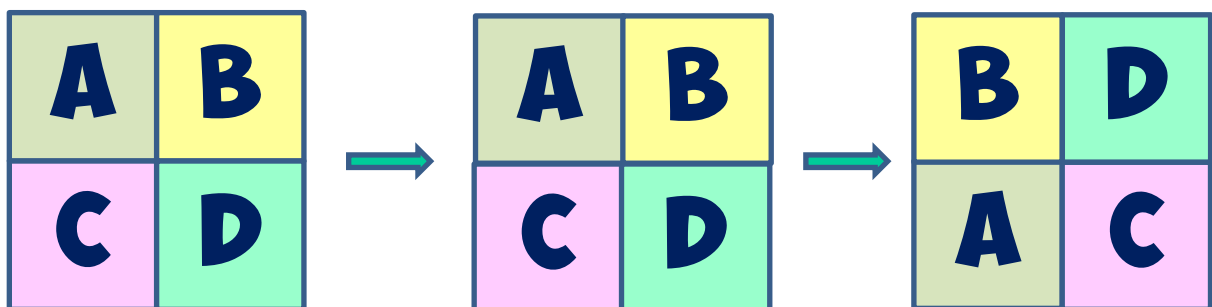
On cherche à écrire une fonction qui effectue la rotation d'une image de 90 degrés en utilisant la stratégie « diviser pour régner ». Le fichier python utilise la bibliothèque **PIL** (*Python Image Library*) et son module `Image`.

```
import PIL.Image as PIL
img=PIL.open("picture.png")
largeur,hauteur=img.size      # dimensions de l'image en pixels
px=img.load()
# si 0<=x<largeur et 0<=y<hauteur,
# la couleur du pixel (x,y) est donnée par px[x,y]
# on peut modifier la couleur d'un pixel (x,y) avec l'affectation
# px[x,y]=c où c est une couleur
PIL.Image.show(img)          # affiche l'image
```

On suppose que l'image est carrée et que sa dimension est une puissance de 2, par ex. 1024*1024.

L'idée consiste

- à découper l'image en 4,
- à effectuer la rotation de 90 degrés de chacun des morceaux, puis à les déplacer vers leur position finale.



- ou bien à les déplacer vers leur position finale puis à effectuer la rotation de 90 degrés de chacun des morceaux.

